

Ex No: 2d

IMPLEMENTATION OF FORWARD CHAINING

Aim:

To implement forward Chaining.

Procedure:

Program:

Facts: The knowledge base containing information about facts

```
facts = {  
    'a': True,  
    'b': True,  
    'c': False  
}
```

Rules: List of rules where each rule is a tuple (conclusion, conditions)

```
rules = [  
    ('d', ['a', 'b']), # d can be concluded if a and b are true  
    ('e', ['b', 'c']), # e can be concluded if b and c are true  
    ('f', ['d', 'e']) # f can be concluded if d and e are true  
]
```

Forward Chaining Function

```
def forward_chaining(facts, rules, goal):
```

```

inferred = set(facts.keys()) # Initialize inferred facts with the given facts
new_inferred = set(facts.keys()) # Start with the facts
while new_inferred:
    current_inferred = set() # Set to hold newly inferred facts
    for rule in rules:
        head, body = rule
        if head not in inferred and all(fact in inferred for fact in body):
            current_inferred.add(head)
            if current_inferred:
                inferred.update(current_inferred)
            new_inferred = current_inferred # Set new inferred facts for the next iteration
        else:
            new_inferred = set() # No new facts can be inferred

    return goal in inferred # Check if the goal is in the inferred facts

# Example Usage
goal = 'f'

if forward_chaining(facts, rules, goal):
    print(f"The goal '{goal}' can be achieved.")
else:
    print(f"The goal '{goal}' cannot be achieved.")

```

Output:

The goal 'f' can be achieved.

The goal 'e' cannot be achieved.

The goal 'd' can be achieved.

Output:

The goal 'f' cannot be achieved.

The goal 'e' cannot be achieved.

The goal 'd' can be achieved.